String Class in Java

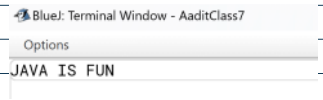
23 January 2025 20:19

Java String

In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java String. For example:

```
char A[] = {'J', 'A', 'V', 'A', ' ', 'I', 'S', ' ', 'F', 'U', 'N'};
System.out.println(A);
```

Output:



String str = new String(A); //here char A[] data is converted to String type.

str

"JAVA IS FUN" (String)

Java String class provides lot of methods to perform operations on strings such as compare(), concat(), equals(), split(), length(), compareTo(), substring etc.

What is String in Java?

Generally, string is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The java.lang.String class is used to create String object.

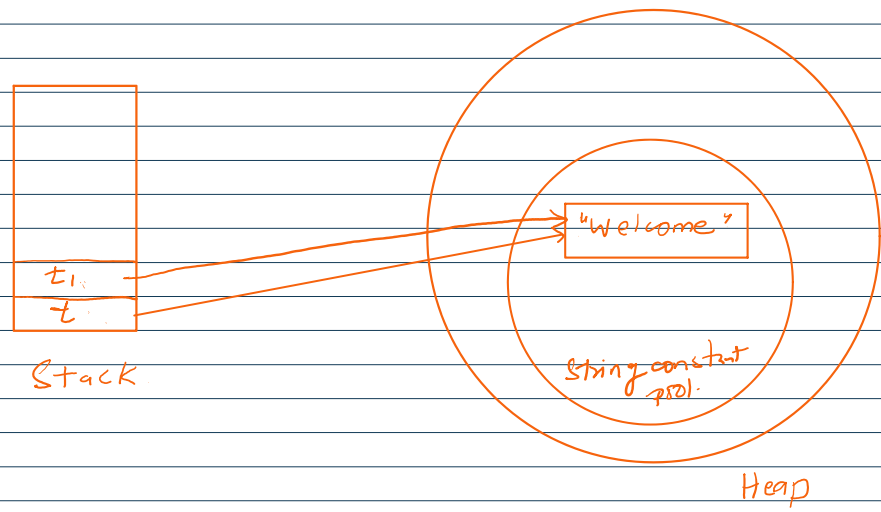
How to create string object?

There are two ways to create String object:

1. **By String Literal**
2. **By new keyword**

1. String Literal:

```
String t = "Welcome";
t
"Welcome"   (String)
```




```
String t = "Welcome";
t
"Welcome"   (String)
String t1 = "Welcome";
t1
"Welcome"   (String)
t
"Welcome"   (String)
```

Each time you create a string literal, the JVM checks the <u>"string constant pool"</u> first. If those string already exist in the pool, a reference to the pooled

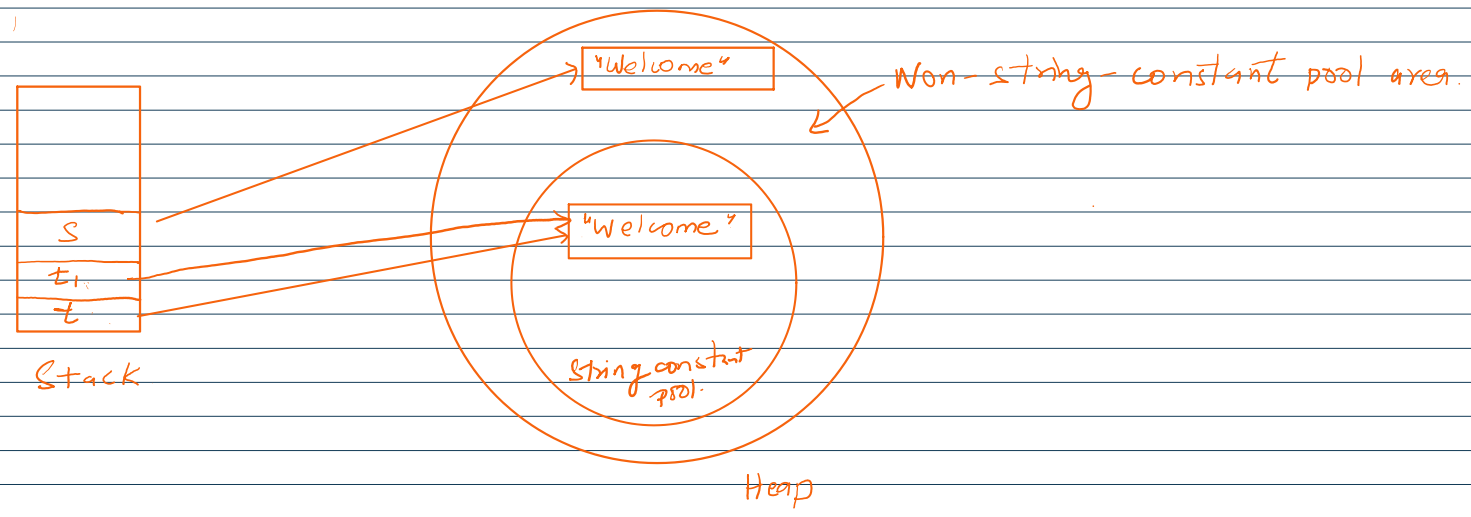
instance is returned. If the string does not exist in the pool, a new string instances created and place in the pool.
In the above example, only one object will be created. Firstly, JVM will not find any string object with the value welcome in the string constant pool. That is why it will create a new object. After that, it will find the string with the value welcome in the pool. It will not create a new object, but will return the reference to the same instance.

Why Java uses the concept of String literal?
To make Java more memory efficient (because no new objects are created if it already in the string constant pool).

2. By new Keyword

```
String s = new String("Welcome");
s
"Welcome"   (String)
```

In such case, JVM will create a new string object in normal (non-pool) heap memory and the literal welcome will be placed in the string constant pool. The variable S will refer to the object in a heap (non-pool)



String: The string class is one of the most fundamental type of in Java, designed to represent immutable sequences of characters.

- Immutability: Once instantiated a string object cannot be modified. This immutable design is a deliberate choice to ensure thread safety, consistency and efficiency, especially, regarding the string pool mechanism.

- String pool: Java maintains a pool of string letters to help save memory. When a new string literal is created, Java checks the pool for a matching strength. If found the new variable references the pooled string, if not the new string is added to the pool.

- Java String class methods
- char charAt(int index)=> returns char value for the particular index.

```
String str = "Java Programming";
str
"Java Programming"   (String)
str.charAt(0);
char c = str.charAt(0);
c
```

```
'J'   (char)
c = str.charAt(1);
c
'a'   (char)
c = str.charAt(2);
c
'v'   (char)
c = str.charAt(4);
c
' '   (char)
c = str.charAt(5);
c
'P'   (char)
c = str.charAt(55);
Exception: java.lang.StringIndexOutOfBoundsException (Index 55 out of bounds for
length 16)

int len = str.length();
len
16   (int)
```

- int length(): It returns string length, and counting starts from 1.

For example:

```
package StringProgramming;
import java.util.*;


/**
 * Write a description of class StringDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringDemo
{
    public static void main(String args[]){
        String str;
        Scanner sv = new Scanner(System.in);
        System.out.print("\fEnter any string: ");
        str = sv.nextLine();
        int len = str.length();
        int i;
        for(i = 0; i < len; i++){
            char c = str.charAt(i);
            System.out.print("\n" + c + " at index: " + i);
        }
    }
}
```

String concat(String str): It concatenates the specified string.
"+" operator in Java is overloaded to concatenate the strings

Example:

```
package StringProgramming;
import java.util.*;


/**
 * Write a description of class StringConcat here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringConcat
```

```
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        String fname, mname, lname;
        System.out.print("\fEnter first name: ");
        fname = sc.next();
        System.out.print("Enter middle name: ");
        mname = sc.next();
        System.out.print("Enter last name: ");
        lname = sc.next();
        //String fullName = fname.concat(" ").concat(mname).concat("
").concat(lname);
        String fullName = fname + " " + mname + " " + lname;
        System.out.print("\nFull name: " + fullName);

    }
}
```

```
BlueJ: Terminal Window - AaditClass7
Options
Enter first name: Mohandas
Enter middle name: Karamchand
Enter last name: Gandhi

Full name: Mohandas Karamchand Gandhi
```

String comparison:

```
String x = "abc";
String y = "abc":
package StringProgramming;

/**
 * Write a description of class StringComparisonDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringComparisonDemo
{
    public static void main(String args[]){
        System.out.print("\f");
        String x = "abc";
        String y = "abc";
        String m = "ABC";
        String z = new String("abc");
        System.out.print((x == y)); //true because memory allocation is same
        System.out.print("\n"+(x == z)); //false because memory is allocated outside
the String pool
        //"==" is comparing memory reference not data. So we need to use String
functions
        System.out.print("\n"+ (x.equals(z))); //true, because it compares data
        System.out.print("\n"+ (x.equals(m))); //flase, because it compares data
        System.out.print("\n" + (x.equalsIgnoreCase(m))); //true; because cases are
ignored
    }
}
```

"equals()" method checks equality of string with given object. And, it is case sensitive.

"equalsIgnoreCase()" method checks equality of string with given String object. And, it ignores the cases.

"compareTo()" method compares and returns integer value. Comparison is done using the conception of arrangement in dictionary in lexicographic manner(alphabetic order). (A < a … Z < z), comparison is done based on Unicode values.

```
int firstString.compareTo(secondString)
```

1. If (firstString > secondString) it return value greater than 0.
2. If (firstString < secondString) it return value less than 0
3. If (firstString = secondString) it return value 0.

```
"ABC".compareTo("AAC")
```

B > A

"compareToIgnoreCase()" works similar to compareTo() method the only difference is it ignores the cases of alphabets. Here 'A' and 'a' is same.

For example:

```
package StringProgramming;

/**
 * Write a description of class StringComparisonDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringComparisonDemo
{
    public static void main(String args[]){
        System.out.print("\f");
        String x = "abc";
        String y = "abc";
        String m = "ABC";
        String z = new String("abc");
        System.out.print((x == y)); //true because memory allocation is same
        System.out.print("\n"+(x == z)); //false because memory is allocated outside
the String pool
        //"==" is comparing memory reference not data. So we need to use String
functions
        System.out.print("\n"+ (x.equals(z))); //true, because it compares data
        System.out.print("\n"+ (x.equals(m))); //flase, because it compares data
        System.out.print("\n" + (x.equalsIgnoreCase(m))); //true; because cases are
ignored

        String name1 = "Aadit";
        String name2 = "Aarav";
        String name3 = "Arnav";
        System.out.print("\nString comparison using compareTo(): ");
        System.out.print("\n" + name1.compareTo(name2));
        System.out.print("\n" + name2.compareTo(name3));
        String name4 = "AADIT";
        System.out.print("\n" + name1.compareToIgnoreCase(name4)); //0
    }
}
```

String toUpperCase() : it returns a string in upper case.

String toLowerCase(): it return a string in lower case.

For example:

```
        String name1 = "Aadit";
        String name4 = "AADIT";
```

```
        System.out.print("\nUpper case: " + name1.toUpperCase());
        System.out.print("\nLower case: " + name4.toLowerCase());
```

```
Upper case: AADIT
Lower case: aadit
```

startsWith() and endsWith():

The method startsWith() checks whether the string starts with the letter passed as arguments. And endsWith () method checks whether the string ends with the letter passed as an argument. Both method returns Boolean value(true/false).

```
public class SearchStringsInJava
{
    public static void main(String args[]){
        String s = "Sachin";
        System.out.print("\f" + s.startsWith("Sa")); //true
        System.out.print("\n"+s.endsWith("in")); //true
        System.out.print("\n"+s.endsWith("iN")); //false
    }
}
```

Output:

```
BlueJ: Terminal Window - AaditClass7
Options
true
true
false
```

The valueOf() method

The String class valueOf() method converts given type such as int, long, float, double, char and char array into String.

```
public class ConvertToString
{
    public static void main(String args[]){
        String s = String.valueOf(3.1416); //double to string
        String PI = s;
        System.out.print("\fPI = " + s);

        char A[] = {'1', '$', '2','$'};
        String t = String.valueOf(A);
        System.out.print("\n" + t);
    }
}
```

OUTPUT:

```
BlueJ: Terminal Window - AaditClass7
Options
PI = 3.1416
1$2$
```

The replace() method

The string class replays () method replaces all occurrence of the first sequence of character with the second sequence of characters.

```
public class ReplacingString
{
    public static void main(String args[]){
        String str = "Java is a programming language. Java is a platform. Java is an
island.";
        String p = str.replaceAll("Java", "Python");
        System.out.print("\fOriginal String: "+ str);
        System.out.print("\nAfter replacing 'Java' with 'Python': " + p);
    }
}
```

OUTPUT:

```
BlueJ: Terminal Window - AaditClass7
Options
Original String: Java is a programming language. Java is a platform. Java is an island.
After replacing 'Java' with 'Python': Python is a programming language. Python is a platform. Python is an island.
```

The trim() method: it eliminates white spaces before and after the String.

```
        String name = "     Shiv    Mishra     ";
        String nm = name.trim();
        System.out.print("\fString with leading and trailing spaces: " + name);
        System.out.print("\nString without leading and trailing spaces: "+nm);
```

OUTPUT:

```
BlueJ: Terminal Window - AaditClass7
Options
String with leading and trailing spaces:     Shiv    Mishra
String without leading and trailing spaces: Shiv    Mishra
```

Function Overloading: means the name of the method is same but it differs with

1. Parameter data type

   OR

2. Number of parameters

rule-1 is implement.

static String	valueOf(boolean b) Returns the string representation of the boolean argument.
static String	valueOf(char c) Returns the string representation of the char argument.
static String	valueOf(char[] data) Returns the string representation of the char array argument.
static String	valueOf(char[] data, int offset, int count) ← rule-2 is implemented Returns the string representation of a specific subarray of the char array argument.
static String	valueOf(double d) Returns the string representation of the double argument.
static String	valueOf(float f) Returns the string representation of the float argument.
static String	valueOf(int i) Returns the string representation of the int argument.
static String	valueOf(long l) Returns the string representation of the long argument.
static String means class reference.	valueOf(Object obj) Returns the string representation of the Object argument.

Function overloading is the feature of Polymorphism in Java. Poly means "multiple" and morphism means "forms".

Another example of Overloading functions:

String	substring(int beginIndex)

Another example of Overloading functions:

String → object reference.	substring(int beginIndex) Returns a string that is a substring of this string.
String	substring(int beginIndex, int endIndex) ← endIndex − 1 Returns a string that is a substring of this string.

```
String str = "JAVA PROGRAMMING";
             0 1 2 3 4 5 6 7 8 9 10 11
String p = str.substring(5); // "PROGRAMMING"
String p1 = str.substring(5, 12); // "PROGRAM"
```

char[]	toCharArray() Converts this string to a new character array.

```
String str = "JAVA PROGRAMMING";
char K[] = str.toCharArray();
```

https://docs.oracle.com/javase/8/docs/api/java/lang/String.html

Use the above link to read about String class in detail. And, find your required method and use also.

If 'ch' or 'str' is not present in the String, then it will return '-1'.

int	indexOf(int ch) Returns the index within this string of the first occurrence of the specified character.
int	indexOf(int ch, int fromIndex) Returns the index within this string of the first occurrence of the specified character, starting the search at the specified index.
int	indexOf(String str) Returns the index within this string of the first occurrence of the specified substring.
int	indexOf(String str, int fromIndex) Returns the index within this string of the first occurrence of the specified substring, starting at the specified index.

Overloaded

int	lastIndexOf(int ch) Returns the index within this string of the last occurrence of the specified character.
int	lastIndexOf(int ch, int fromIndex) Returns the index within this string of the last occurrence of the specified character, searching backward starting at the specified index.
int	lastIndexOf(String str) Returns the index within this string of the last occurrence of the specified substring.
int	lastIndexOf(String str, int fromIndex) Returns the index within this string of the last occurrence of the specified substring, searching backward starting at the specified index.

Home-work:

VI. Java Programming:

1. Write a program to accept two different characters and display the sum and difference of their ASCII values.

   Sample Input: A
   d

   Sample Output: The sum of ASCII values = 165
   The difference of ASCII value = 35

2. Write a program to input an alphabet in upper case or in lower case. Display the next alphabet accordingly. (i.e.: 'a' follows 'b', ... 'z' follows 'a').

3. Write a program to accept a character. If it is a letter, then display the case (i.e.: lower or upper case), otherwise check whether it is a digit or a special character.

   Sample Input: p

   Sample Output: p is a letter.
   p is in lower case.

4. Write a program in Java to accept a word/a String and display the new string after removing all the vowels present in it.

   Sample Input: COMPUTER APPLICATIONS

   Sample Output: CMPTR PPLCTNS

5. Write a program in Java to accept a word/a string. Count all the alphabets excluding vowels present in the word/string and display the result.

   Sample Input: Happy New Year

   Sample Output: No. of alphabets excluding vowels = 8